

M1 — Identity Service + Real Dev Login

Goal: replace the web app's local mock authentication with a real, end-to-end login flow through the Gateway and Identity Service, without disturbing the domains that are still on mock data.

What shipped

Two cohesive Python services plus the web wiring to consume them.

1. Identity Service (`services/identity/`)

FastAPI service, the **sole minter** of primary tokens (ADR-007):

- RS256 JWTs (private key held only here; public JWKS published for verifiers).
- Pluggable providers behind an `AuthProvider` protocol; ships the **Dev Identity Provider** (seeded pilot identities, no credential check).
- Tokens carry **identity/session claims only** — never permissions (ADR-019).
- The dev provider is **structurally impossible in production**: the config layer forces it off when `ENVIRONMENT=production`, and `/auth/dev/*` return 404.
- 29 tests (JWT round-trip/tamper/expiry, provider, prod guard, HTTP surface).

2. Gateway (`apps/gateway/`)

FastAPI BFF, the single browser edge:

- Delegates all token work to Identity — **never mints** anything itself.
- Stores the minted JWT in an **httpOnly gabiél_session cookie**; the browser only ever receives a session view, never the raw token.
- Translation seam between Identity's `snake_case` contract and the browser's `camelCase` types (accepts `{ userId }`, calls Identity with `{ user_id }`).
- CORS locked to the exact web origin with credentials.
- 10 tests (login sets httpOnly cookie, token never in body, session read, logout clears cookie, 401 paths, camelCase translation).

3. Web app (`apps/web/`)

- New **per-domain mock flag** (`isMock('auth')` , env `NEXT_PUBLIC_LIVE_AUTH`).
A single global `USE MOCK` would have forced un-migrated domains (chat, agents, documents...) to call endpoints that don't exist yet; the per-domain flag lets **auth go live while everything else stays on mock**.
- `auth.ts` now routes login/session/logout through the Gateway when auth is live. `gatewayRequest` already sends `credentials: 'include'`.

Topology

```
Browser → Gateway (:8080) → Identity (:8081)    # auth (live in M1)
        → Gateway → SDK → Core                  # resources/agents (M2+)
```

Auth flow (live)

1. `GET /auth/dev/principals` — Gateway proxies Identity's seeded identities.
2. `POST /auth/dev/login { userId }` — Gateway → Identity `/auth/dev/login { user_id }` → Identity mints a JWT → Gateway sets the `httpOnly` cookie and returns the `Session` view.
3. `GET /auth/session` — Gateway reads the cookie → Identity `/auth/introspect` → returns `Session`, or 401.
4. `POST /auth/logout` — Gateway clears the cookie (204).

Boundary guarantees upheld

- Identity is the only token minter; Gateway only delegates + manages the cookie.
- JWTs carry identity/session data only — no permissions.
- Dev provider cannot run in production.
- The browser never talks to Identity/Core directly, and never handles a token.

Verification

- `services/identity` : `python -m pytest` → 29 passed.
- `apps/gateway` : `python -m pytest` → 10 passed.
- `apps/web` : `pnpm typecheck` → clean.
- E2E: logged in as Alice through the live chain; dashboard rendered with the real session; `document.cookie` does **not** expose `gabriel_session` (`httpOnly` confirmed); `/auth/session` returned 200; logout → 401. Chat/agents/documents still render mock data (per-domain flag confirmed).

Run the stack

```
./scripts/dev.sh          # Identity :8081, Gateway :8080, web :3000 (live auth)
```

See `apps/web/.env.local.example` for the web flags.

Not in M1 (next)

- **M2**: Gateway → SDK → Core for resources/agents; WebSocket brokering; retire the remaining mock providers domain by domain.